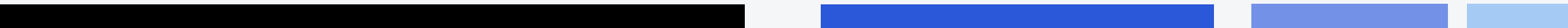


Lecture-1 1

Virtual Functions



Dynamic Polymorphism

Dynamic Polymorphism lets a single function call behave differently depending on the actual object that receives it. It is achieved through inheritance plus virtual functions.

- **Compile-time polymorphism:** function/operator overloading, templates
- **Run-time polymorphism:** virtual functions through base-class pointers/references

Access through a base pointer

```
class Base {
public:
    void show() { cout << "Base::show\n"; }    // NOT virtual
};
class Derived : public Base {
public:
    void show() { cout << "Derived::show\n"; }
};
int main() {
    Derived d;
    Base* bp = &d;
    bp->show();           // prints: Base::show    (!)
    d.show();            // prints: Derived::show
}
```

Access through a base pointer

Why?

- Without virtual, the call is bound at COMPILE time using the pointer's STATIC type (Base*).
- The compiler does not look inside the actual object. It just calls Base::show().
- Fix: declare show() as virtual in Base.

Polymorphism

```
class Shape {
public:
    virtual void draw() {cout << "Shape\n";} };
class Circle : public Shape {
public:
    void draw() override {cout << "Circle\n";} };
class Square : public Shape {
public:
    void draw() override {cout << "Square\n";} };
int main() {
    Shape* s;   Circle c;   Square sq;
    s = &c;     s->draw();   // Circle
    s = &sq;    s->draw();   // Square
}
```

Virtual Functions

- A function the compiler resolves at run time, not compile time
- Declared in the base class with the keyword `virtual`.
- Once virtual in the base, it stays virtual in every derived class.
- Lets a base-class pointer call the correct derived version.

```
class Base {  
public:  
    virtual void show() { cout << "Base::show\n"; } };  
class Derived : public Base {  
public:  
    void show() override { cout << "Derived::show\n"; }  
};
```

Early (Static) vs Late (Dynamic) Binding

Early / Static Binding

- Decision made at compile time.
- Default for non-virtual functions.
- Faster, direct function call.
- Used by overloading & templates.

Late / Dynamic Binding

- Decision made at run time.
- Triggered by virtual functions.
- Slightly slower (vtable lookup).
- Enables true polymorphism.
- Needed for base-pointer dispatch.

Pure Virtual Functions & Abstract Classes

- A function with no body. The derived classes MUST override it
- Declared as: `virtual void f() = 0;` (the '`= 0`' makes it pure)
- A class with at least one pure virtual function is ABSTRACT.
- You cannot create an object of an abstract class.
- You CAN have pointers/references of the abstract type.
- Derived class becomes concrete only when ALL pure virtuals are overridden.

Pure Virtual Functions & Abstract Classes

```
class Shape {
public:
    virtual double area() const = 0;};
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
// Shape s;           // ERROR: cannot instantiate
Shape* p = new Circle(5);
```

Virtual Destructors

- If you delete a derived object through a base pointer and the base destructor is NOT virtual, only the base destructor runs. Derived resources leak.
- Always make the destructor virtual in a polymorphic base class

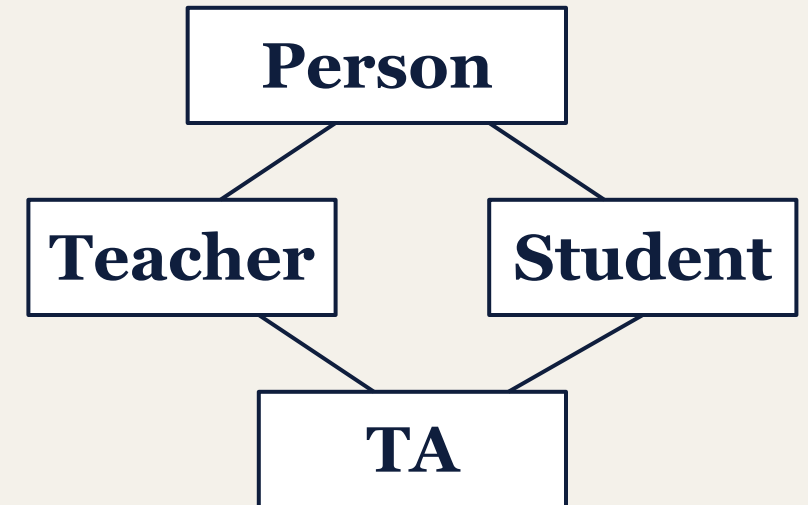
Virtual Destructors

```
class Base {
public:
    Base()          { cout << "Base ctor\n"; }
    ~Base()         { cout << "Base dtor\n"; };
class Derived : public Base {
public:
    Derived()       { cout << "Derived ctor\n"; }
    ~Derived()      { cout << "Derived dtor\n"; };
int main() {
    Base* p = new Derived();
    delete p;    // prints: Base ctor / Derived ctor / Base dtor
}               // Fix: make Base's destructor 'virtual'.
```

Multiple Inheritance & The Diamond Problem

When a class inherits from two classes that both inherit from a common base, the base members can appear twice — this is the diamond problem.

- TA inherits Person twice is ambiguity.
- Solution: declare the common base as virtual.
- `class Teacher : virtual public Person { ... };`
- `class Student : virtual public Person { ... };`
- Now TA contains ONE shared Person sub-object.



Virtual Base Class — Code

```
class Person {  
public:  
    string name; };  
class Teacher : virtual public Person { // virtual  
public:  
    int salary; };  
class Student : virtual public Person { // virtual  
public:  
    int rollNo; };  
class TA : public Teacher, public Student {};  
int main() {  
    TA t;  
    t.name = "Ali";}
```

Friend Functions

- Declared inside a class with the keyword friend.
- Defined OUTSIDE the class — it is NOT a member.
- Not called using object.func() syntax; called like a regular function.
- Useful when an operation needs private data of TWO different classes.
- Does not get the implicit 'this' pointer.

Friend Functions

```
class Box {  
    int width;  
public:  
    Box(int w) : width(w) {}  
    friend void printWidth(const Box& b); // declaration  
};  
  
void printWidth(const Box& b) {           // not a member  
    cout << "Width = " << b.width;      // private OK  
}  
  
int main() { Box b(10); printWidth(b); } // prints: Width = 10
```

Friend Classes

- Whole class B can be a friend of class A.
- Every member function of B can then see A's private/protected members.
- Friendship is NOT inherited, NOT transitive.
- Use sparingly as it weakens encapsulation.

Friend Classes

```
class Engine {  
    int rpm = 800;  
    friend class Car;    // Car can see Engine's privates  
};  
  
class Car {  
    Engine e;  
public:  
    void show() { cout << "rpm=" << e.rpm; }    // legal  
};
```

Static Data Members & Static Functions

- static data member: a SINGLE shared variable, no matter how many objects exist.
- Must be defined OUTSIDE the class (one definition rule).
- static member function: belongs to the class, not to any object.
- Has no 'this' pointer; can only touch static members directly.

Static Data Members & Static Functions

```
class Counter {  
    static int count;  
public:  
    Counter() { count++; }  
    static int getCount() { return count; }  
};  
int Counter::count = 0;
```

Accessing Static Functions

```
int main() {  
    Counter c1, c2, c3;  
  
    cout << Counter::getCount();    // prints: 3  
  
    cout << c1.getCount();          // prints: 3  
  
    // Inside a static function, this is ILLEGAL:  
    // void Counter::reset() { count = 0; this->count = 0; }  
}
```

Assignment vs Copy Initialization

- Assignment: `a3 = a2;` calls `operator=`
- Copy init.: `alpha a3 = a2;` calls copy constructor (NOT `operator=`)
- Copy init.: `alpha a3(a2);` same thing, different syntax
- By default, the compiler provides BOTH and they do member-by-member copy.
- Override them when your class manages a resource (memory, file, socket).

Rule of three: if you define one of { destructor, copy constructor, `operator=` } you almost always need all three.

Overloading the Assignment Operator

```
class alpha {  
    int data;  
public:  
    alpha(int d = 0) : data(d) {}  
  
    alpha& operator=(const alpha& a) {  
        cout << "Assignment operator invoked\n";  
        data = a.data;  
        return *this; //enables a3=a2= a1;  
    }  
};
```

- operator= is NOT inherited.

The Copy Constructor

- Signature: `ClassName(const ClassName& other);`
- Argument **MUST** be a reference as passing by value would call itself forever.
- Called for: `T a = b;` `T a(b);` passing by value; returning by value.

The Copy Constructor

```
class alpha {
    int data;
public:
    alpha(int d = 0) : data(d) {}
    alpha(const alpha& a) {                // copy constructor
        cout << "Copy ctor invoked\n";
        data = a.data;
    };
void f(alpha x) {}                        // copy ctor (pass by value)
int main() {
    alpha a1(7);
    alpha a2 = a1;                        // copy initialization -> copy ctor
    f(a1);                                // copy ctor again
}
```

How to Prohibit Copying

- Declare both operator= and the copy constructor as PRIVATE.
- Any copy attempt becomes a compile-time error.

How to Prohibit Copying

```
class alpha {  
private:  
    alpha& operator=(const alpha&);    // disabled  
    alpha(const alpha&);              // disabled  
public:  
    alpha() {}  
};  
  
class beta {  
public:  
    beta() = default;  
    beta(const beta&) = delete;  
    beta& operator=(const beta&) = delete;  
};
```

The 'this' Pointer

- Inside any non-static member function, 'this' points to the current object.
- Used to disambiguate parameters from members: `this->x = x;`
- Used to return `*this` which enables chaining and reference-return idioms.
- Static member functions do NOT have 'this'.

```
class alpha {  
    int data;  
public:  
    alpha& operator=(const alpha& a) {  
        data = a.data;  
        return *this;  
    }  
};
```

Dynamic Type Information (RTTI)

- Two operators: `dynamic_cast<T*>(p)` and `typeid(expr)`.
- Require the base class to be POLYMORPHIC with at least one virtual function.
- Your compiler must have RTTI enabled (default in most compilers).
- `dynamic_cast` returns `nullptr` on failure for pointers, throws `bad_cast` for references.

```
class Animal      { public: virtual ~Animal() {} };
class Dog : public Animal { public: void bark() {} };
void example(Animal* a) {
    if (Dog* d = dynamic_cast<Dog*>(a))        // safe down-cast
        d->bark();
    cout << typeid(*a).name();                // run-time type name
}
```

Quick Recap — Common Bug Patterns

Bug	Symptom	Fix
Base method not virtual	bp->show() calls Base version	Add 'virtual' in base
No virtual destructor	Derived dtor never runs	virtual ~Base()
Diamond inheritance	Ambiguous base member	Use virtual inheritance
Static member not defined	Linker error	int Class::var = 0; in .cpp
this in static function	Compile error	Static functions have no 'this'

Multiple Choice Questions

1. Which keyword enables runtime polymorphism in C++?

- ☐ A static
- ☐ B virtual
- ☐ C friend
- ☐ D const

2. A class containing at least one pure virtual function is called...

- A** A friend class
- B** A static class
- C** An abstract class
- D** A virtual class

3. If the base destructor is not virtual and we delete via a base pointer to a derived object...

- A** Both destructors run in order
- B** Only the derived destructor runs
- C** Only the base destructor runs and derived resources leak
- D** It causes a compile error

4. A friend function of class A...

- A** Is a member of A and has 'this'
- B** Cannot access private members of A
- C** Is non-member but may access private members of A
- D** Must be declared in every derived class of A

5. A static member function of class C...

- A** Has access to the 'this' pointer
- B** Can be called as C::f() with no object
- C** Is shared per object, not per class
- D** Cannot exist without a static data member

6. Which statement invokes the COPY CONSTRUCTOR rather than operator=?

- A** `a = b;`
- B** `alpha a; a = b;`
- C** `alpha a = b;`
- D** `swap(a, b);`

7. Why is the diamond problem solved by virtual inheritance?

- A** It removes virtual functions from the base
- B** It causes the derived class to skip the base
- C** It ensures only ONE shared sub-object of the common base exists
- D** It makes all base members private

8. `dynamic_cast` on a pointer fails when...

- A** It throws `bad_cast` immediately
- B** It returns `nullptr`
- C** It silently performs the cast anyway
- D** The program aborts at compile time

Short Questions

Coding Q1 — Predict the output

```
class A {  
public: void f() { cout << "A "; }  
};  
class B : public A {  
public: void f() { cout << "B "; }  
};  
int main() {  
    A a; B b;  
    A* p = &b;  
    a.f();  
    b.f();  
    p->f();  
}
```

Coding Q1 — Predict the output

```
class A {  
public: void f() { cout << "A "; }  
};  
class B : public A {  
public: void f() { cout << "B "; }  
};  
int main() {  
    A a; B b;  
    A* p = &b;  
    a.f();  
    b.f();  
    p->f();  
}
```

Answer

Output: A B A

f() is NOT virtual, so p->f() binds at compile time to A::f().

Coding Q2 — Make it polymorphic

```
class A {  
public: _____ void f() { cout << "A "; }  
};  
class B : public A {  
public: void f() { cout << "B "; }  
};  
int main() {  
    B b;  
    A* p = &b;  
    p->f();  
    b.f();  
}
```

output: B B.

Coding Q2 — Make it polymorphic

```
class A {  
public: ____ void f() { cout << "A "; }  
};  
class B : public A {  
public: void f() { cout << "B "; }  
};  
int main() {  
    B b;  
    A* p = &b;  
    p->f();  
    b.f();  
}
```

Answer

Fill the blank with 'virtual'.

The call `p->f()` now uses the vtable and prints B; `b.f()` also prints B → output: B B.

Coding Q3 — Static members

```
class Counter {  
    static int n;  
public:  
    Counter() { ____ ; }  
    static int get() { return ____ ; }  
};  
// missing line outside the class:  
// _____  
  
int main() {  
    Counter a, b, c;  
    cout << Counter::get();  
}
```

Coding Q3 – Static members

```
class Counter {  
    static int n;  
public:  
    Counter() { ____ ; }  
    static int get() { return ____ ; }  
};  
// missing line outside the class:  
// _____  
  
int main() {  
    Counter a, b, c;  
    cout << Counter::get();  
}
```

Answer

Constructor body: n++;

Return value: n;

Outside the class:

int Counter::n = 0;

Coding Q4 — Copy constructor

```
class alpha {  
    int data;  
public:  
    alpha(alpha a) {           // <-- error here  
        data = a.data;  
    }  
};
```

Coding Q4 — Copy constructor

```
class alpha {  
    int data;  
public:  
    alpha(alpha a) {           // <--  
error here  
        data = a.data;  
    }  
};
```

```
alpha(const alpha& a)  
{  
    data = a.data;  
}
```

Answer

Passing 'a' by VALUE would itself invoke the copy constructor → infinite recursion. The argument must be a reference: `alpha(const alpha& a) { ... }`

Coding Q5 — Will Derived's destructor run?

```
class Base {
public:
    ~Base()    { cout << "~Base "; }
    // not virtual
};
class Derived : public Base {
public:
    ~Derived() { cout << "~Derived "; }
};
int main() {
    Base* p = new Derived();
    delete p;
}
```

Coding Q5 — Will Derived's destructor run?

```
class Base {  
public:  
    ~Base()    { cout << "~Base "; }  
    // not virtual  
};  
class Derived : public Base {  
public:  
    ~Derived() { cout << "~Derived "; }  
};  
int main() {  
    Base* p = new Derived();  
    delete p;  
}
```

Answer

Output: ~Base

Derived's destructor is skipped because Base::~~Base is not virtual.

Fix: virtual ~Base();

Coding Q6 — Write a friend function

```
class Account {  
    double balance;  
public:  
    Account(double b) : balance(b) {}  
    // 1) Declare friend here  
};  
  
// 2) Define the friend function below  
//     so the call show(a) prints the balance.  
int main() {  
    Account a(2500.75);  
    show(a);  
}
```

Coding Q6 — Write a friend function

```
class Account {  
    double balance;  
public:  
    Account(double b) : balance(b) {}  
    // 1) Declare friend here  
};  
  
// 2) Define the friend function below  
//     so the call show(a) prints the  
//     balance.  
int main() {  
    Account a(2500.75);  
    show(a);  
}
```

```
// (1)  
    friend void show(  
        const Account&);  
};  
// (2)  
void show(const  
Account& a) {  
    cout << "Balance = "  
        << a.balance;  
}
```

Coding Q7 — Abstract class

Why does line marked () fail to compile?*

```
class Animal {
public:
    virtual void speak() = 0;
};
class Cat : public Animal {
public:
    void speak() override { cout <<
    "Meow"; }
};
int main() {
    Animal a;           // (*) ERROR
    Cat c;  c.speak();
}
```

Coding Q7 — Abstract class

Why does line marked () fail to compile?*

```
class Animal {
public:
    virtual void speak() = 0;
};
class Cat : public Animal {
public:
    void speak() override { cout <<
    "Meow"; }
};
int main() {
    Animal a;           // (*) ERROR
    Cat c;  c.speak();
}
```

Answer

Animal has a pure virtual function (= 0) so it is ABSTRACT. You cannot create an instance of it.

Legal:

```
Animal* p = new Cat();
```